



< Previous



Next >

Using randomness

Bookmark this page

Question: Does the definition of randomness prove useful beyond mathematics?

Answer: Yes. For instance for random number generation and in cryptography.

A successful theory of randomness has many applications. The most immediate one is the generation of pseudo-random sequences.
Pseudo-random numbers can be generated using the following program.

```
$ python
>>> import random
>>> print(random.randint(0, 10**32))
53217645600250283088552254822697
>>> print(random.randint(0, 10**32))
42682293524798380206208898923066
```

Pseudo random numbers

1/1 point (ungraded)

Why are pseudo-random numbers generated by a computer considered "pseudo" random? (several answers are acceptable)

- ☐ Because the computer's memory is limited, so the pseudo-random number generation will eventually loop.
- ☐ Because the pseudo-random generation uses the computer's clock, which is not random.
- ☒ Because the pseudo-random generation program has a small size.
- ☐ Because the pseudo-random generation program itself is deterministic.

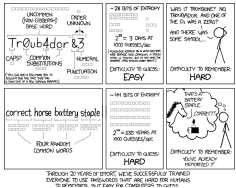
✓

Answer
Correct:
Pseudo-random numbers are not really random. This means that they are compressible. This can be seen by considering the way they are computed. For reproducibility reasons, all pseudo-random algorithms use a *seed* S , i.e. a number used to initialize the generation procedure. Then, the algorithms feeds itself with new seeds computed from the previous one. When no seed is provided, Python uses the output of the system's clock. The complexity of any pseudo-random number R can be bounded above by $C(R) \leq C(S) + C(r)$, where r is the rank of R in the generation procedure. On Python3, the clock provides a 12 digit number. If one has no clue about the current time, then $C(S) \leq \log_2(10^{12}) \approx 40$, which is less than $\log_2(10^{32}) \approx 106$. And of course, one may know much about the higher weight digits returned by the clock.

Submit You have used 1 of 1 attempt

✓ Correct (1/1 point)

Randomness theory has immediate applications in cryptography. Consider for instance the design of robust passwords. A good password should be easy to remember but hard to guess (see [Chapter 1](#)). At face value, the two requirements are contradictory. To reconcile them, you must target the difference between you (human cognitive equipment + cultural knowledge + specific knowledge) and the machine



RSA

0/1 point (ungraded)

The RSA encryption system is based on the difficulty of factoring the product m of two large numbers p and q , *i.e.* the difficulty of retrieving p and q such that $m = p \times q$. One of the numbers p and q is necessary to decode messages. What is the best estimate of the complexity of a binary message M of length k , when only m and the encrypted version of M based on m are available?

- ☐ 0
- ☐ $\min(\log p, \log q)$
- ☐ $\log \log m + 1$
- ☐ $(\log m) / 2$
- ☐ $\log k$
- ☒ $\log k + \log m$

✖

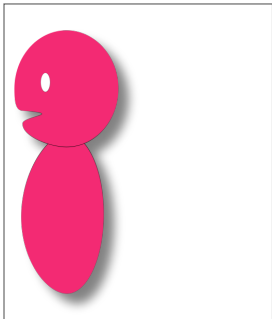
Answer
Incorrect:
The encrypted version of M + the knowledge of m is enough information to retrieve M , regardless of the computational efforts you need to retrieve p and q . So 0 is the correct answer.

Submit

You have used 1 of 1 attempt

ⓘ Answers are displayed within the problem

So you agree with me that numbers can be really random.





edX

- About
- Affiliates
- edX for Business
- Open edX
- Careers
- News

Legal

- Terms of Service & Honor Code
- Privacy Policy
- Accessibility Policy
- Trademark Policy
- Sitemap
- Cookie Policy
- Your Privacy Choices

Connect

- Idea Hub
- Contact Us
- Help Center
- Security
- Media Kit

